

we use the `_context.Entry()` method to get the `EntityEntry` object of the `invoice`. Then, we set the `State` property to `EntityState.Modified`.

In EF Core, each instance of `DbContext` has a `ChangeTracker` to track the changes in the entities, which is a powerful feature of EF Core. In other words, EF Core knows the state of each entity – whether it is `added`, `deleted`, or `modified`. When we update the entity, we just update the entity in the memory. EF Core can track the changes. When the `SaveChangesAsync()` method is called, it will generate the SQL statement to update the data in the database.

The `EntityState` enum can have the following values:

- `Detached`: The entity is not being tracked by the context.
- `Unchanged`: The entity is being tracked by the context, but the values are not changed.
- `Deleted`: The entity is being tracked and exists in the database, but it has been marked for deletion. So, when the `SaveChangesAsync()` method is called, EF Core will generate the SQL statement to delete the entity from the database.
- `Modified`: The entity is being tracked and exists in the database, and the values have been modified in `DbContext`. When the `SaveChangesAsync()` method is called, EF Core will generate the SQL statement to update the entity in the database.
- `Added`: The entity is being tracked but it does not exist in the database. When the `SaveChangesAsync()` method is called, EF Core will generate the SQL statement to insert the entity into the database.

```
// Update only the properties that have changed _context.  
Entry(invoiceToUpdate).CurrentValues.SetValues(invoice);
```

From EF Core 7.0, we have a new method called `ExecuteDeleteAsync()` that can be used to delete an entity without loading it first. The code is as follows:

```
await _context.Invoices.Where(x => x.Id == id).ExecuteDeleteAsync();
```

Important note

The `ExecuteDeleteAsync()` method does not involve the change tracker, so it will execute the SQL statement immediately to the database. It does not need to call the `SaveChangesAsync()` method at the end. It is a recommended way to delete one entity (or more) from EF Core 7.0 and later versions. However, if the entity already exists in `DbContext` and is tracked by the change tracker, executing the SQL statement directly may cause the data in `DbContext` and the database to be inconsistent. In this case, you may need to use the `Remove()` method or set the `EntityState` property to `Deleted` to delete the entity from `DbContext`. Please consider your scenario carefully before using the `ExecuteDeleteAsync()` method.

Activate
Free Edition

If a property is named `Id` or `<entity name>Id`, EF Core will map it to the primary key.

If EF Core detects that the relationship **between two models** is **one-to-many**, it will map the **navigation** property **to a foreign key column** in the database automatically.

The annotation way for model is:

```
using JLokaTestEFCore.enums;
using Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace JLokaTestEFCore.Models
{
    [Table("Invoices")]
    public class Invoice
    {
        [Column("Id")]
        [Key]
        public Guid Id { get; set; }
        [Column(name: "InvoiceNumber", TypeName = "varchar(32)")]
        [Required]
        public string InvoiceNumber { get; set; } = string.Empty;
        [Column(name: "ContactName")]
        [Required]
        [MaxLength(32)]
        public string ContactName { get; set; } = string.Empty;
        [Column(name: "Description")]
        [MaxLength(256)]
        public string? Description { get; set; }

        [Column("Amount")]
        [Precision(18, 2)]
        [Range(0, 999999999999.99)]
        public decimal Amount { get; set; }
        [Column(name: "InvoiceDate", TypeName = "datetimeoffset")]
        public DateTimeOffset InvoiceDate { get; set; }
        [Column(name: "DueDate", TypeName = "datetimeoffset")]
        public DateTimeOffset DueDate { get; set; }
        [Column(name: "Status", TypeName = "varchar(16)")]
        public InvoiceStatus Status { get; set; }
    }
}
```

Fluent API is a set of extension methods that you can use to configure the mappings gracefully. It is the most flexible and powerful way to apply the mapping configuration without polluting the model classes. Another important thing to note is that Fluent API has a higher priority than data annotations. If you configure the same property in both data annotations and Fluent API, Fluent API will override the data annotations.

```

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // the seeder data ...

    // using Fluent API
    modelBuilder.Entity<Invoice>(b =>
    {
        b.ToTable("Invoices");
        b.HasKey(i => i.Id);
        b.Property(p => p.Id).HasColumnName("Id");
        b.Property(p =>
p.InvoiceNumber).HasColumnName("InvoiceNumber").HasColumnType("varchar(32)").IsRequired();
        b.Property(p =>
p.ContactName).HasColumnName("ContactName").HasMaxLength(32).IsRequired();
        b.Property(p =>
p.Description).HasColumnName("Description").HasMaxLength(256);
        b.Property(p =>
p.Amount).HasColumnName("Amount").HasColumnType("decimal(18,2)").IsRequired();
        b.Property(p => p.Amount).HasColumnName("Amount").HasPrecision(18,
2);
        b.Property(p =>
p.InvoiceDate).HasColumnName("InvoiceDate").HasColumnType("datetimeoffset").IsRequired();
        b.Property(p =>
p.DueDate).HasColumnName("DueDate").HasColumnType("datetimeoffset").IsRequired();
        b.Property(p =>
p.Status).HasColumnName("Status").HasMaxLength(16).HasConversion(
            v => v.ToString(),
            v => (InvoiceStatus)Enum.Parse(typeof(InvoiceStatus), v)
        );
    });
}

*****

```

In a large project, there may be a lot of model classes. If we put all the mapping configurations in the `OnModelCreating()` method, the method will be very long and hard to maintain. To make the code more readable and maintainable, we can extract the mapping configuration to one class or a couple of separate classes.

One way to do this is to create an extension method for the `ModelBuilder` class.

```

*****

```

Another way to separate the mapping configurations is to implement the *IEntityTypeConfiguration* interface.

```
public class InvoiceConfiguration : IEntityTypeConfiguration<Invoice>
{
    public void Configure(EntityTypeBuilder<Invoice> builder)
```

Configuring the mapping between models and database

```
{
    builder.ToTable("Invoices");
    // Other mapping configurations are omitted for brevity
}
```

Then, in the `OnModelCreating()` method, there are two ways to apply the configuration:

- If you use the `ApplyConfiguration()` method, add the following code to the `OnModelCreating()` method:

```
modelBuilder.ApplyConfiguration(new InvoiceConfiguration());
// You can continue to call the ApplyConfiguration method for
other entities. such as
// modelBuilder.ApplyConfiguration(new
InvoiceItemConfiguration());
```

- Or you can call the `Configure()` method directly:

```
new InvoiceConfiguration().Configure(modelBuilder.
Entity<Invoice>());
// You can continue to call the Configure method for other
entities. such as
// new InvoiceItemConfiguration().Configure(modelBuilder.
Entity<InvoiceItem>());
```

Activate Win

As the project grows, it might be a little bit tedious to call the mapping configuration for each entity. In this case, EF Core has a method called **ApplyConfigurationsFromAssembly()** to apply all the configurations in the assembly, as shown in the following code:

```
modelBuilder.ApplyConfigurationsFromAssembly(typeof(InvoiceDbContext).Assembly);
```

You can choose the one that fits your project best. One reminder is that if you use the **ApplyConfigurationsFromAssembly()** method, you need to make sure that all the configuration classes are in the same assembly as the **DbContext** class. Also, you cannot control the order of the configurations. If the order matters, you need to call each configuration explicitly in the correct order.

Important note

During the migration, the data may get lost if the data type is changed. For example, if the data type is changed from **nvarchar(max)** to **varchar(32)**, **the original data will be truncated to 32 characters**. Please make sure you understand the data type change before you run the migration.

To define one-to-many relationship between **Invoice** and **InvoiceItem**:

```
namespace JLokaTestEFCore.Models
{
    public class InvoiceItem
    {
        public Guid Id { get; set; }
        public string Name { get; set; } = String.Empty;
        public string? Description { get; set; }
        public decimal UnitPrice { get; set; }
        public decimal Quantity { get; set; }
        public decimal Amount { get; set; }
        public Guid InvoiceId { get; set; }
        public Invoice? Invoice { get; set; }
    }
}
```

```

using JLokaTestEFCore.enums;
using Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace JLokaTestEFCore.Models
{
    public class Invoice
    {
        public Guid Id { get; set; }
        public string InvoiceNumber { get; set; } = string.Empty;
        public string ContactName { get; set; } = string.Empty;
        public string? Description { get; set; }
        public decimal Amount { get; set; }
        public DateTimeOffset InvoiceDate { get; set; }
        public DateTimeOffset DueDate { get; set; }
        public InvoiceStatus Status { get; set; }

        public List<InvoiceItem> InvoiceItems { get; set; } = new
List<InvoiceItem>();
    }
}

*****

```

An invoice has a collection of invoice items, and an invoice item belongs to an invoice. It is a one-to-many relationship, where we can identify these terms:

- **Principal entity:** This is the one entity in a one-to-many relationship. It is also called the parent entity. In the preceding case, Invoice is the principal entity.
- **Dependent entity:** This is the many entity in a one-to-many relationship. It is also called the child entity. In the preceding case, InvoiceItem is the dependent entity. It has an InvoiceId foreign key property to identify the parent entity.
- **Principal key:** This is the primary key of the principal entity, which uniquely identifies the principal entity. In the preceding case, the Id property of the Invoice class is the principal key.
- **Foreign key:** The InvoiceId property of the InvoiceItem class is the foreign key, which is used to store the principal key value of the parent entity.
- **Navigation property:** This is used to reference the related entity. It can be defined on the principal or dependent entity. There are two types of navigation properties:
 - **Collection navigation property:** This is defined on the principal entity, which is used to reference a collection of related entities. For example, the InvoiceItems property of the Invoice class is a collection navigation property.

- **Reference navigation property.** This is defined on the dependent entity, which is used to reference a single related entity. For example, the Invoice property of the InvoiceItem class is a reference navigation property.

If we remove InvoiceId from InvoiceItem Model, then when we apply the migration, then the result is:

EF Core can still create a column named InvoiceId in the InvoiceItems table, and apply the foreign key constraint to it, which is called **shadow foreign key property**. This is because EF Core has its built-in convention to do this.

There are a few scenarios where EF Core can discover one-to-many relationships between entities:

- The dependent entity has a **reference navigation property** to the principal entity
- The principal entity **has a collection navigation property** to the dependent entity
- The **reference navigation property and the collection navigation property are included** at both ends
- The **reference navigation property and the collection navigation property are included at both ends**, and the foreign key property is included in the dependent entity

1. To explicitly configure the one-to-many relationship between entities, we can use the `HasOne()`, `WithMany()`, and `HasMany()`, `WithOne()` methods. Add the following code to the `InvoiceConfiguration` class:

```
builder.HasMany(x => x.InvoiceItems)
        .WithOne(x => x.Invoice)
        .HasForeignKey(x => x.InvoiceId);
```

The `HasMany()` method is used to configure the collection navigation property, and the `WithOne()` method is used to configure the reference navigation property. The `HasForeignKey()` method is used to configure the foreign key property. So, the preceding code explicitly configures that one invoice can have many invoice items, and the `InvoiceId` property of the `InvoiceItem` class is the foreign key. If you add a migration now, you will find that EF Core will generate the same code as that generated by the convention.

2. A relationship can be also defined for the `InvoiceItem` class. Remove the preceding configuration code for the `Invoice` class, and add the following code to the `InvoiceItemConfiguration` class:

```
builder.HasOne(i => i.Invoice)
    .WithMany(i => i.InvoiceItems)
    .HasForeignKey(i => i.InvoiceId)
    .OnDelete(DeleteBehavior.Cascade);
```

It should be easy to understand now. The `HasOne()` method is used to configure the reference navigation property, and the `WithMany` method is used to configure the collection navigation property.

Note that we also explicitly configured the `OnDelete()` action to `Cascade`, which is the same as the one generated by the convention. But we can change it to other options if needed. That said, the Fluent API is more flexible than the convention.

As a one-to-many relationship can be defined in either direction, which side should we configure the relationship on? It depends on the scenario. If the two entities have a strong one-to-many relationship, it does not really matter which side we configure the relationship on. But if the two entities are loosely coupled, we had better configure the relationship on the dependent entity.

For example, a `User` entity is shared by many other entities, such as `Post`, `Comment`, `Invoice`, and so on. Each `Post` entity can have an `Author` property that is a reference navigation property to the `User` entity, and `Comment` and `Invoice` also do the same. However, the `User` entity does not need to have the collection navigation properties to the `Post`, `Comment`, and `Invoice` entities. In this case, we should configure the relationship on the `Post`, `Comment`, and `Invoice` entities.

To configure this kind of relationship, we can ignore the parameter for the `WithMany` method, because the `User` entity does not have the collection navigation property to the `Post` entity, as shown in the following code:

```
public void Configure(EntityTypeBuilder<Post> builder)
{
    // Omitted for brevity
    builder.HasOne(x => x.Author)
        .WithMany()
        .HasForeignKey(x => x.AuthorId);
    // Omitted for brevity
}
```

To make swagger or json body ignore the fields that we don't want to include them, when posting data:

```
using JLokaTestEFCore.enums;

namespace JLokaTestEFCore.Models
{
    // [Table("Invoices")]
    public class Invoice
    {
        // [Column("Id")]
        // [Key]
        public Guid Id { get; set; }
        // [Column(name: "InvoiceNumber", TypeName = "varchar(32)")]
        // [Required]
        public string InvoiceNumber { get; set; } = string.Empty;
        // [Column(name: "ContactName")]
        // [Required]
        // [MaxLength(32)]
        public string ContactName { get; set; } = string.Empty;
        // [Column(name: "Description")]
        // [MaxLength(256)]
        public string? Description { get; set; }
        // [Column("Amount")]
        // [Precision(18, 2)]
        // [Range(0, 999999999999.99)]
        public decimal Amount { get; set; }
        // [Column(name: "InvoiceDate", TypeName = "datetimeoffset")]
        public DateTimeOffset InvoiceDate { get; set; }
        // [Column(name: "DueDate", TypeName = "datetimeoffset")]
        public DateTimeOffset DueDate { get; set; }
        // [Column(name: "Status", TypeName = "varchar(16)")]
        public InvoiceStatus Status { get; set; }
        // [JsonIgnore]
        public List<InvoiceItem> InvoiceItems { get; set; } = new
List<InvoiceItem>();
    }
}
```